

Lista de Exercícios

Aula 10 . Polimorfismo

Disciplina	Programação Orientada a Objetos		
Professor	Chauã Queirolo		
Entrega	-	Valor	-

Resumo

Polimorfismo é um conceito da Programação Orientada a Objetos que permite que uma mesma referência, método ou operação tenha comportamentos diferentes dependendo do objeto utilizado. A ideia central é “uma mesma ação, várias formas de execução”.

Exemplo: uma classe abstrata Animal pode ter o método emitirSom(). As classes Cachorro, Gato e Passaro podem sobrescrever esse método, cada uma emitindo um som diferente. O código chama o mesmo método, mas o comportamento muda conforme o objeto.

Os principais tipos de polimorfismo são:

- **Polimorfismo por sobrescrita:** ocorre quando uma classe filha redefine um método herdado da classe pai. É comum em herança e classes abstratas. Em Dart, usamos @override.
- **Polimorfismo por interface:** ocorre quando classes diferentes implementam a mesma interface. Mesmo sendo classes sem relação direta de herança, elas podem ser tratadas pelo mesmo tipo de interface.
- **Polimorfismo paramétrico ou genérico:** ocorre quando uma classe, função ou estrutura trabalha com tipos variados usando generics. Em Dart, isso aparece com <T>.
- **Polimorfismo por sobrecarga:** ocorre quando existem métodos com o mesmo nome, mas com parâmetros diferentes. Em linguagens como Java e C#, isso é comum. Em Dart, não existe sobrecarga clássica de métodos. A alternativa é usar parâmetros opcionais ou nomeados.

Em resumo, o polimorfismo permite escrever código mais flexível e reutilizável, pois o programa pode trabalhar com uma classe pai, interface ou tipo genérico, deixando que cada objeto execute o comportamento adequado.

Atividades

Para os exercícios abaixo, crie uma classe genérica de apoio:

lista_generica.dart

```
class ListaGenerica<T> {  
    final List<T> itens = [];  
  
    void adicionar(T item) {  
        itens.add(item);  
    }  
  
    void imprimirItens() {  
        for (var item in itens) {  
            print(item);  
        }  
    }  
  
    List<T> obterItens() {  
        return itens;  
    }  
}
```

Essa classe deverá ser usada nos exercícios para armazenar objetos por meio da classe pai ou da interface, e não diretamente pelas classes filhas.

Questão 1 – Funcionários

Crie uma classe abstrata `Funcionario` com os atributos `nome` e `salario`. Crie também uma interface `Bonificavel`, com o método `calcularBonus()`.

A classe `FuncionarioComum` deve herdar de `Funcionario` e implementar `Bonificavel`. O bônus do funcionário comum deve ser de 5% do salário.

A classe `Gerente` deve herdar de `Funcionario`, implementar `Bonificavel` e possuir também o atributo `setor`. O bônus do gerente deve ser de 15% do salário.

Sobrescreva o método `toString()` na classe pai `Funcionario` ou nas classes filhas `FuncionarioComum` e `Gerente`. O método deve retornar `nome`, `salário`, `tipo do funcionário` e, quando houver, `setor`.

No main, crie os seguintes 5 objetos:

- Ana, salário 2500.00, funcionário comum.
- Bruno, salário 3200.00, funcionário comum.
- Carla, salário 7000.00, gerente do setor Financeiro.
- Diego, salário 2800.00, funcionário comum.

- Elisa, salário 8500.00, gerente do setor Tecnologia.

Adicione todos os objetos em uma `ListaGenerica<Bonificavel>`.

Chame o método `imprimirltens()` da lista.

Depois, faça um loop no main percorrendo a lista e chamando o método comum da interface `calcularBonus()` para cada objeto.

Questão 2 – Produtos

Crie uma classe `Produto` com os atributos `nome` e `preco`. Crie uma interface `Exibivel`, com o método `exibir()`.

Crie a classe `ProdutoPerecivel`, que herda de `Produto`, implementa `Exibivel` e possui o atributo `dataValidade`.

Crie também a classe `ProdutoEletronico`, que herda de `Produto`, implementa `Exibivel` e possui o atributo `garantiaMeses`.

Sobrescreva o método `toString()` nas classes filhas. O método deve retornar nome, preço e os dados específicos de cada produto.

No main, crie os seguintes 5 objetos:

- Leite, preço 6.50, validade 20/05/2026.
- Iogurte, preço 4.80, validade 18/05/2026.
- Notebook, preço 3500.00, garantia de 24 meses.
- Smartphone, preço 2200.00, garantia de 12 meses.
- Queijo, preço 28.90, validade 25/05/2026.

Adicione todos os objetos em uma `ListaGenerica<Exibivel>`.

Chame o método `imprimirltens()` da lista.

Depois, faça um loop no main percorrendo a lista e chamando o método comum da interface `exibir()` para cada objeto.

Questão 3 – Veículos

Crie uma classe abstrata `Veiculo` com os atributos `marca`, `modelo` e `ano`. Crie uma interface `Fichavel`, com o método `exibirFicha()`.

Crie a classe `Carro`, que herda de `Veiculo`, implementa `Fichavel` e possui o atributo `quantidadePortas`.

Crie a classe `Moto`, que herda de `Veiculo`, implementa `Fichavel` e possui o atributo `cilindradas`.

Crie a classe `Caminhao`, que herda de `Veiculo`, implementa `Fichavel` e possui o atributo `capacidadeCarga`.

Sobrescreva o método `toString()` na classe pai `Veiculo` ou nas classes filhas. A impressão deve mostrar marca, modelo, ano, tipo do veículo e os dados específicos.

No main, crie os seguintes 5 objetos:

- Toyota Corolla, ano 2022, carro com 4 portas.
- Honda Civic, ano 2021, carro com 4 portas.
- Yamaha Fazer, ano 2023, moto de 250 cilindradas.
- Honda Biz, ano 2020, moto de 125 cilindradas.
- Volvo VM, ano 2019, caminhão com capacidade de 12 toneladas.

Adicione todos os objetos em uma `ListaGenerica<Fichavel>`.

Chame o método `imprimirItens()` da lista.

Depois, faça um loop no main percorrendo a lista e chamando o método comum da interface `exibirFicha()` para cada objeto.

Questão 4 – Contas bancárias

Crie uma classe `ContaBancaria` com os atributos titular e saldo. Crie uma interface `Rentavel`, com o método `aplicarRendimento()`.

Crie a classe `ContaPoupanca`, que herda de `ContaBancaria` e implementa `Rentavel`. A conta poupança deve aplicar rendimento de 1%.

Crie a classe `ContaCorrente`, que herda de `ContaBancaria` e implementa `Rentavel`. A conta corrente deve aplicar rendimento de 5%.

Crie também a classe `ContaInvestimento`, que herda de `ContaBancaria` e implementa `Rentavel`. A conta investimento deve aplicar rendimento de 8%.

Sobrescreva o método `toString()` na classe pai `ContaBancaria` ou nas classes filhas. O método deve retornar titular, saldo e tipo da conta.

No main, crie os seguintes 5 objetos:

- Ana, conta poupança, saldo 1000.00.
- Bruno, conta corrente, saldo 1500.00.
- Carla, conta investimento, saldo 5000.00.
- Diego, conta poupança, saldo 2500.00.
- Elisa, conta corrente, saldo 800.00.

Adicione todos os objetos em uma `ListaGenerica<Rentavel>`.

Chame o método `imprimirItens()` da lista antes de aplicar o rendimento.

Depois, faça um loop no main percorrendo a lista e chamando o método comum da interface `aplicarRendimento()` para cada objeto.

Ao final, chame novamente o método `imprimirItens()` para mostrar os saldos atualizados.

Questão 5 – Pessoas

Crie uma classe abstrata `Pessoa` com os atributos nome e idade. Crie uma interface `Apresentavel`, com o método `exibirDados()`.

Crie a classe Aluno, que herda de Pessoa, implementa Apresentavel e possui os atributos matricula e curso.

Crie a classe Professor, que herda de Pessoa, implementa Apresentavel e possui os atributos disciplina e salario.

Crie também a classe Coordenador, que herda de Pessoa, implementa Apresentavel e possui os atributos area e tempoExperiencia.

Sobrescreva o método toString() nas classes filhas. A impressão deve mostrar nome, idade, tipo da pessoa e os dados específicos.

No main, crie os seguintes 5 objetos:

- Ana, 19 anos, aluna, matrícula A001, curso ADS.
- Bruno, 22 anos, aluno, matrícula A002, curso Engenharia de Software.
- Carla, 38 anos, professora, disciplina POO, salário 2000.00.
- Diego, 41 anos, professor, disciplina Banco de Dados, salário 1900.00.
- Elisa, 45 anos, coordenadora, área Tecnologia, 15 anos de experiência.

Adicione todos os objetos em uma ListaGenerica<Apresentavel>.

Chame o método imprimirltens() da lista.

Depois, faça um loop no main percorrendo a lista e chamando o método comum da interface exibirDados() para cada objeto.

Questão 6 – Ingressos

Crie uma classe Ingresso com os atributos evento e valor. Crie uma interface Calculavel, com o método valorFinal().

Crie a classe IngressoVip, que herda de Ingresso, implementa Calculavel e possui o atributo valorAdicional.

Crie a classe IngressoPromocional, que herda de Ingresso, implementa Calculavel e possui o atributo desconto.

Crie a classe IngressoCamarote, que herda de Ingresso, implementa Calculavel e possui os atributos taxaServico e localizacao.

Sobrescreva o método toString() nas classes filhas. O método deve retornar evento, valor original, tipo do ingresso e valor final.

No main, crie os seguintes 5 objetos:

- Show Rock, ingresso VIP, valor 200.00, adicional 80.00.
- Teatro Infantil, ingresso promocional, valor 100.00, desconto 20.00.
- Festival Jazz, ingresso VIP, valor 300.00, taxa de serviço 50.00, localização Setor A.
- Cinema Especial, ingresso promocional, valor 60.00, desconto 15.00.
- Show Pop, ingresso VIP, valor 180.00, adicional 70.00.

Adicione todos os objetos em uma `ListaGenerica<Calculavel>`.

Chame o método `imprimirltens()` da lista.

Depois, faça um loop no main percorrendo a lista e chamando o método comum da interface `valorFinal()` para cada objeto.

Questão 7 – Animais

Crie uma classe abstrata `Animal` com o atributo `nome`. Crie uma interface `Sonoro`, com o método `emitirSom()`.

Crie as classes `Cachorro`, `Gato`, `Passaro`, `Vaca` e `Ovelha`, todas herdando de `Animal` e implementando `Sonoro`.

Cada classe deve implementar o método `emitirSom()` de forma específica.

Sobrescreva o método `toString()` na classe pai `Animal` ou nas classes filhas. A impressão deve mostrar o nome do animal e o tipo do animal.

No main, crie os seguintes 5 objetos:

- Rex, cachorro.
- Mimi, gato.
- Piu, pássaro.
- Mimosa, vaca.
- Dolly, ovelha.

Adicione todos os objetos em uma `ListaGenerica<Sonoro>`.

Chame o método `imprimirltens()` da lista.

Depois, faça um loop no main percorrendo a lista e chamando o método comum da interface `emitirSom()` para cada objeto.

Questão 8 – Livros

Crie uma classe `Livro` com os atributos `titulo` e `autor`. Crie uma interface `Informativo`, com o método `exibirInformacoes()`.

Crie a classe `LivroDigital`, que herda de `Livro`, implementa `Informativo` e possui o atributo `tamanhoArquivo`.

Crie a classe `LivroFisico`, que herda de `Livro`, implementa `Informativo` e possui o atributo `quantidadePaginas`.

Crie a classe `Audiobook`, que herda de `Livro`, implementa `Informativo` e possui o atributo `duracaoMinutos`.

Sobrescreva o método `toString()` nas classes filhas. O método deve retornar título, autor, tipo do livro e os dados específicos.

No main, crie os seguintes 5 objetos:

- Clean Code, Robert Martin, livro físico, 425 páginas.

- Dart Básico, Mariana Souza, livro digital, 12 MB.
- POO na Prática, Carlos Lima, audiobook, 180 minutos.
- Algoritmos Modernos, Ana Torres, livro físico, 350 páginas.
- Flutter Essencial, Pedro Alves, livro digital, 25 MB.

Adicione todos os objetos em uma `ListaGenerica<Informativo>`.

Chame o método `imprimirItens()` da lista.

Depois, faça um loop no main percorrendo a lista e chamando o método comum da interface `exibirInformacoes()` para cada objeto.

Questão 9 – Pedidos

Crie uma classe abstrata `Pedido` com os atributos `codigo` e `valorTotal`. Crie uma interface `Processavel`, com os métodos `valorFinal()` e `exibirPedido()`.

Crie a classe `PedidoDelivery`, que herda de `Pedido`, implementa `Processavel` e possui os atributos `enderecoEntrega` e `taxaEntrega`.

Crie a classe `PedidoRetirada`, que herda de `Pedido`, implementa `Processavel` e possui o atributo `nomeCliente`.

Crie a classe `PedidoExpress`, que herda de `Pedido`, implementa `Processavel` e possui o atributo `taxaUrgencia`.

Sobrescreva o método `toString()` nas classes filhas. O método deve retornar código, valor total, tipo do pedido, dados específicos e valor final.

No main, crie os seguintes 5 objetos:

- Pedido 1001, delivery, valor 120.00, endereço Rua A, 100, taxa 15.00.
- Pedido 1002, retirada, valor 80.00, cliente Ana.
- Pedido 1003, express, valor 200.00, taxa de urgência 40.00.
- Pedido 1004, delivery, valor 150.00, endereço Rua B, 250, taxa 20.00.
- Pedido 1005, retirada, valor 60.00, cliente Bruno.

Adicione todos os objetos em uma `ListaGenerica<Processavel>`.

Chame o método `imprimirItens()` da lista.

Depois, faça um loop no main percorrendo a lista e chamando os métodos comuns da interface `valorFinal()` e `exibirPedido()` para cada objeto.

Questão 10 – Dispositivos

Crie uma classe `Dispositivo` com os atributos `marca` e `modelo`. Crie uma interface `Ligavel`, com os métodos `ligar()` e `desligar()`.

Crie a classe `Smartphone`, que herda de `Dispositivo`, implementa `Ligavel` e possui o atributo `sistemaOperacional`.

Crie a classe Notebook, que herda de Dispositivo, implementa Ligavel e possui o atributo memoriaRam.

Crie a classe Televisao, que herda de Dispositivo, implementa Ligavel e possui o atributo tamanhoPolegadas.

Sobrescreva o método toString() nas classes filhas. O método deve retornar marca, modelo, tipo do dispositivo e os dados específicos.

No main, crie os seguintes 5 objetos:

- Samsung Galaxy S24, smartphone, sistema Android.
- Apple iPhone 15, smartphone, sistema iOS.
- Dell Inspiron, notebook, 16 GB de RAM.
- Lenovo ThinkPad, notebook, 32 GB de RAM.
- LG OLED55, televisão, 55 polegadas.

Adicione todos os objetos em uma ListaGenerica<Ligavel>.

Chame o método imprimirltens() da lista.

Depois, faça um loop no main percorrendo a lista e chamando os métodos comuns da interface ligar() e desligar() para cada objeto.

Questão 11 – Personagens

Crie uma pequena hierarquia de classes usando herança, classes abstratas e interfaces em Dart. A atividade deve explorar a diferença entre classe abstrata e interface: as classes abstratas devem representar conceitos gerais do sistema, enquanto as interfaces devem representar comportamentos obrigatórios.

Crie as interfaces Atacavel, Magico e Flamejante, conforme a proposta original do exercício de personagens, que define comportamentos de ataque, magia e fogo .

Crie a classe abstrata Personagem com os atributos nome, vida e nivel. Ela deve possuir os métodos receberDano(), estaVivo() e exhibirStatus().

Crie a classe abstrata Combatente, que herda de Personagem e implementa Atacavel.

Crie a classe abstrata Inimigo, que herda de Personagem e possui o atributo recompensa.

Crie as classes Guerreiro, Arqueiro, Mago, Goblin e Dragao.

O Guerreiro deve herdar de Combatente. O Arqueiro deve herdar de Combatente. O Mago deve herdar de Combatente e implementar Magico. O Goblin deve herdar de Inimigo. O Dragao deve herdar de Inimigo e implementar Flamejante.

Sobrescreva o método toString() na classe pai Personagem ou nas classes filhas. A impressão deve mostrar nome, vida, nível, tipo do personagem e atributos específicos, como força, armadura, flechas, mana, velocidade ou poder de fogo.

No main, crie os seguintes 5 objetos:

- Thoran, guerreiro, vida 120, nível 5, força 20, armadura 15.
- Lia, arqueira, vida 80, nível 4, força 15, flechas 10.

- Merlin, mago, vida 70, nível 6, força 10, mana 50.
- Gob, goblin, vida 40, nível 2, recompensa 100, velocidade 25.
- Ignis, dragão, vida 200, nível 10, recompensa 1000, poder de fogo 35.

Adicione os personagens combatentes em uma `ListaGenerica<Atacavel>`. Adicione os personagens mágicos em uma `ListaGenerica<Magico>`. Adicione os personagens flamejantes em uma `ListaGenerica<Flamejante>`. Adicione todos os personagens em uma `ListaGenerica<Personagem>`.

Chame o método `imprimirltens()` da lista de personagens.

Depois, faça os seguintes loops no main:

- Percorra a `ListaGenerica<Atacavel>` e chame o método comum da interface `atacar()`.
- Percorra a `ListaGenerica<Magico>` e chame o método comum da interface `lançarMagia()`.
- Percorra a `ListaGenerica<Flamejante>` e chame o método comum da interface `soltarFogo()`.

Ao final, chame novamente `imprimirltens()` na lista de personagens para verificar o estado atualizado de cada objeto.

Respostas

Questão 1

bonificavel.dart

```
abstract class Bonificavel {  
    double calcularBonus();  
}
```

funcionario.dart

```
abstract class Funcionario {  
    String nome;  
    double salario;  
  
    Funcionario(this.nome, this.salario);  
  
    @override  
    String toString() {  
        return 'Nome: $nome | Salário: R\$ ${salario.toStringAsFixed(2)}';  
    }  
}
```

funcionario_comum.dart

```
import 'funcionario.dart';
import 'bonificavel.dart';

class FuncionarioComum extends Funcionario implements Bonificavel {
  FuncionarioComum(String nome, double salario) : super(nome, salario);

  @override
  double calcularBonus() {
    return salario * 0.05;
  }

  @override
  String toString() {
    return 'Funcionário Comum | Nome: $nome | Salário: R\$ ${salario.toStringAs-
Fixed(2)} | Bônus: R\$ ${calcularBonus().toStringAsFixed(2)}';
  }
}
```

gerente.dart

```
import 'funcionario.dart';
import 'bonificavel.dart';

class Gerente extends Funcionario implements Bonificavel {
  String setor;

  Gerente(String nome, double salario, this.setor) : super(nome, salario);

  @override
  double calcularBonus() {
    return salario * 0.15;
  }

  @override
  String toString() {
    return 'Gerente | Nome: $nome | Salário: R\$ ${salario.toStringAsFixed(2)} |
Setor: $setor | Bônus: R\$ ${calcularBonus().toStringAsFixed(2)}';
  }
}
```

main.dart

```
import 'bonificavel.dart';
import 'funcionario_comum.dart';
import 'gerente.dart';
import 'lista_generica.dart';

void main() {
  ListaGenerica<Bonificavel> funcionarios = ListaGenerica<Bonificavel>();

  Bonificavel funcionario1 = FuncionarioComum('Ana', 2500.00);
  Bonificavel funcionario2 = FuncionarioComum('Bruno', 3200.00);
  Bonificavel funcionario3 = Gerente('Carla', 7000.00, 'Financeiro');
  Bonificavel funcionario4 = FuncionarioComum('Diego', 2800.00);
  Bonificavel funcionario5 = Gerente('Elisa', 8500.00, 'Tecnologia');

  funcionarios.adicionar(funcionario1);
  funcionarios.adicionar(funcionario2);
  funcionarios.adicionar(funcionario3);
  funcionarios.adicionar(funcionario4);
  funcionarios.adicionar(funcionario5);

  print('--- Impressão dos funcionários ---');
  funcionarios.imprimirItens();

  print('\n--- Bônus dos funcionários ---');
  for (var funcionario in funcionarios.obterItens()) {
    print('Bônus: R$ ${funcionario.calcularBonus().toStringAsFixed(2)}');
  }
}
```